VERIQO Findings Closure Report

Round: F (Findings) **Date:** 2026-09-04 **Branch:** `claude/l99-gap-coverage-nv70zy` **Input:** `Findings.docx`, written after reading the Constitutional Sequencing deliverable.

---

0. The finding that mattered most, and it was about this repository's own documents

The review made one observation that no test in this repository would have caught, and it was correct:

<!-- RUNTIME-LEDGER: historical -->

> Closure report yang searchable masih menampilkan artefak runtime lama: AUDIT-007 qualification_begun / AUDIT-008 proof_attached / AUDIT-009 resolved / AUDIT-010 proof.sealed

(The stream quoted above is the **superseded** one the reviewer found in those documents. It is not what `evidence/RUNTIME_EVIDENCE.json` holds; the current stream is in §1.)

The committed `evidence/RUNTIME_EVIDENCE.json` was right. Two documents in `docs/` were not. A reader grepping the repository found the stale prose first and had no way to tell it from current fact.

This is worth stating plainly because it is a different class of defect from the previous round's. The sequencing defect was wrong *code*. This was correct code with wrong *documentation*, which is the failure mode a system like VERIQO is least equipped to notice, because every test it runs is a test of the code.

0.1 What the two hits actually were

Document	Line	Verdict
`docs/VERIQO_SISA_BACKLOG_CLOSURE_REPORT.md`	126–130	**A real defect.** It presented the ten-event, unlawfully-ordered stream as the current runtime evidence.
`docs/VERIQO_CONSTITUTIONAL_SEQUENCING_AUDIT.md`	41–44	**A legitimate historical quotation** — §1.1, "What the artefact actually said" — but nothing marked it as such, so a grep could not distinguish it from the first.

The second is not excused by being intentional. If a search cannot tell a historical quotation from a current claim, the quotation is a defect in practice whatever it was in intent.

0.2 The fix is a test, not an edit

Correcting the two documents would have fixed today's instance and left the mechanism intact. The verification the reviewer performed by hand — *does every document that quotes the runtime ledger agree with the ledger that is committed?* — is mechanical, so it now runs on every build:

`test/architecture/docs_traceability_test.go`, **`TestNoDocumentMisquotesTheRuntimeLedger`**

It reads every markdown file under `docs/`, extracts every `AUDIT-NNN` citation, and compares it against `evidence/RUNTIME_EVIDENCE.json` — the *committed file*, not a fresh run, because a generator that emits a lawful stream while a stale artefact sits in git is precisely the failure being guarded against.

Run against the repository as the reviewer found it, the new test reports the following — every line below quotes a **superseded** citation, which is the point of the output:

<!-- RUNTIME-LEDGER: historical -->

```
docs/VERIQO_CONSTITUTIONAL_SEQUENCING_AUDIT.md:41 cites AUDIT-007 as "qualification_begun";
    the committed artefact records "qualification.reverse_closed" at that index
```

Six such lines in all — three in each of the two documents, at exactly the positions the reviewer found by reading. (Only the first is reproduced here: the rule is strict enough that a longer sample would itself have to be marked historical line by line, which is the test working as intended on this very report.)

0.3 Why a superseded report is not rewritten

A dated closure report is a record of what a round produced. Editing its ledger block to match today's artefact would make it claim something that round did not achieve — a worse dishonesty than the one being fixed.

So the rule is not *every document must match*. It is **every document must match, or must say out loud that it is quoting history**, using a marker that sits next to the quotation:

```
<!-- RUNTIME-LEDGER: historical -->
```

`TestTheHistoricalMarkerIsActuallyLoadBearing` proves the escape hatch is narrow: a marker more than eight lines above a citation does not license it. A marker parked at the top of a long document would otherwise excuse every stale citation below it, which is the same as having no rule.

The Sisa Backlog report now carries, immediately above its block, a paragraph beginning **"SUPERSEDED — read this before the block below"** that names the defect in the stream it shows and points at the current artefact. The Constitutional Sequencing report's §1.1 block now labels each line `<- SUPERSEDED`.

---

1. The runtime artefact, as committed

The reviewer asked to be shown that the *committed* file, on the latest branch, actually holds the lawful order. It does, and it now holds one record more than it did:

```
001 case.opened                 008 case.qualification_begun
002 case.scoped                 009 case.proof_attached
003 case.evidence_pinned        010 proof.sealed
004 case.hypothesis_recorded    011 claim.finding_founded
005 case.claim_registered       012 case.decision_authorized
006 case.hypothesis_tested      013 case.resolved
007 qualification.reverse_closed
                                014 redaction.derivative_released
```

Record 014 is new this round and is discussed in §3: it is the Article 18 pipeline running on a live path.

`TestAStaleCitationIsRecognisedAsStale` asserts, against the committed file, that record 007 is the reverse closure and record 013 is the resolution — so if the sequencing fix were ever reverted, this test fails on the artefact rather than on a document.

---

2. The harder Finding invariant

The review asked for more than an unexported constructor:

> Every Finding MUST have: exactly one ProofObjectID, exactly one CaseID, exactly one authority path, immutable lineage.

2.1 What was already true, and what was not

Requirement	Before	Now
Exactly one ProofObjectID	held (single `proofHash` field, set only by `NewFinding`)	unchanged, plus verified by `VerifyIntegrity`

Requirement	Before	Now
Exactly one CaseID	**not enforced** — `NewFinding` never checked it was non-empty	`ErrFindingWithoutCase`
Exactly one authority path	**not recorded at all**	`FindingAuthorityPath`, covered by the hash
Immutable lineage	held in-process (unexported fields, copying accessors)	plus `VerifyIntegrity` for a finding that crossed a boundary

2.2 The authority path

```
const FindingAuthorityPath = "proof.Seal -> proof.deriveSufficiency -> proof.NewFinding"
```

It is written by the only constructor and **covered by the finding's hash**. `TestTheAuthorityPathIsCoveredByTheHash` proves that: forging the path changes the hash, so a finding claiming to have come from `caseproofgraph.BuildFinding` fails `VerifyIntegrity` rather than passing as a finding with an unusual provenance note.

This is the object-level answer to the review's larger point in §4 below: "who was allowed to say this?" now travels *with the object*, not in a document about the object.

2.3 An honest note on reachability

`ErrFindingWithoutCase` and `ErrFindingWithoutProposition` cannot be reached through `Seal`, because `Seal` already refuses an object with no scope or no proposition. They are the second line, for an object deserialized from a snapshot or reconstructed by a future caller, where `Seal`'s refusal did not travel with the bytes.

Rather than assert an unreachable branch, the tests reach it honestly: `sealedBypassingValidation` hand-computes a valid canonical hash over an object that skipped validation — modelling the only way such an object can exist — and the checks then fire. This is recorded because a test that asserts a branch it cannot reach is a test that passes forever.

2.4 What the graph can no longer do

The review's list was: CANNOT CREATE, CANNOT MODIFY, CANNOT CHANGE STANCE, CANNOT CHANGE SUFFICIENCY.

`caseproofgraph.Build` now:

- takes findings as a **parameter** (done last round),
- calls `f.VerifyIntegrity()` before rendering one — a finding that lost agreement with its own hash is refused rather than drawn as fact,
- refuses a finding belonging to **another case**, not merely another proof object,
- reads every node attribute off the finding, and records `authority_path` among them, so the graph *reports* the authority instead of implying it holds it.

---

3. Article 18, on a live path

The review's demand was specific: build the workers, produce a real derivative, verify it byte-level, hash it, link its provenance, emit a ledger event — and make failure mean `NO_RELEASE`, never `warning → release`.

3.1 The trap that would have made this worthless

PDF, XLSX and PPTX all compress their content.

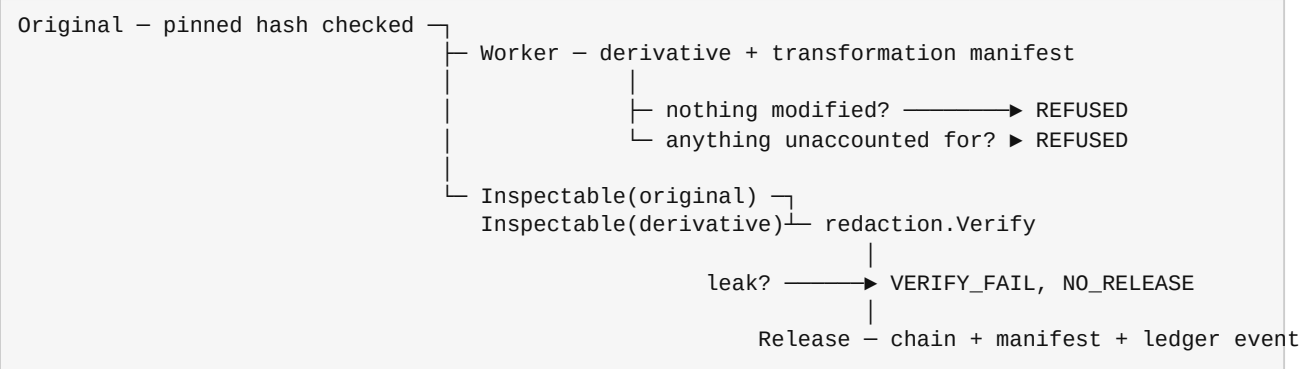
A forbidden term sitting in plain sight in a spreadsheet cell does not appear anywhere in the `.xlsx` file's bytes, because those bytes are deflated. A pipeline built the obvious way — hand the container to `redaction.Verify` — would report

absence for **every** document, including one where nothing was removed. That is a check which cannot fail, which is worse than no check, because it produces a signed record saying it passed.

So verification never runs against the container. `worker.Inspectable` returns the concatenation of every decompressed part **and** the container bytes; the second half covers `STORE`d zip entries, PDF metadata outside content streams, and terms hiding in part filenames.

`TestCompressionWouldHaveHiddenTheTerm` is the guard on the guard. For each of the three formats it asserts that the term is genuinely **absent** from the raw container and **present** in the inspectable view. If a fixture ever stops compressing, that test fails and says so, rather than every other test in the package quietly becoming weaker.

3.2 The pipeline



`Release` has no exported constructor. `Pipeline.Run` returns one only after the verifier passed, so **holding a Release is the statement that verification succeeded** — the outcome cannot be separated from its evidence.

3.3 What each worker does

- **XLSX / PPTX** (`ooxml.go`) — unzips the package, rewrites the XML parts (including `xl/sharedStrings.xml`, where a workbook's cell text actually lives), and rebuilds the archive with a fixed modification time so the derivative is deterministic. `TestTheDerivativeIsDeterministic` proves two runs produce identical bytes: an evidence version whose hash changes with the clock cannot be reproduced by a third party.
- **PDF** (`pdf.go`) — inflates FlateDecode content streams, rewrites the text, re-deflates, fixes every `/Length`, and **rebuilds the cross-reference table** so the result opens. `TestTheRedactedPDFIsStructurallyValid` walks the rebuilt xref and checks each in-use entry lands on the `N 0 obj` it indexes. A redaction that removes the term by corrupting the file removes the evidence with it.

3.4 What the workers refuse, and why refusing is the feature

A redactor that silently skips what it does not understand is worse than no redactor: it emits a document carrying a provenance record that says the content was removed. So the PDF worker enumerates what it recognises but cannot process, and **any** of them present means `ErrRefused`:

Refused	Because
`/Encrypt`	it cannot read the streams, so must not report them clean
more than one `%%EOF`	incremental updates keep earlier revisions in the file; a term redacted in the latest is recoverable from a previous one
`/ObjStm`	objects compressed inside another object, not unpacked
`/XRef` stream	a cross-reference stream rather than a table
`LZW`, `DCT`, `CCITT`, `JBIG2`, ASCII filters, `/Crypt`	filters it does not decode

The OOXML workers refuse a binary attachment (a PNG, an embedded workbook) that carries a forbidden term, naming the part.

Three more refusals close the ways a clean report could be manufactured:

- **A worker that changed nothing** is refused before the verifier runs. Otherwise a document with nothing to remove would acquire a redaction provenance record.
- **A redaction with no terms** is refused. A copy is not a redaction.
- **A marker containing a forbidden term** is refused. Replacing `Acme` with `[REDACTED: Acme]` removes nothing.

3.5 Case and escaping

Two ways a worker could pass its own verifier while leaving the term readable, both closed:

- **Case.** The verifier checks a case-folded encoding, so a worker removing `Acme Holdings Ltd` and leaving `ACME HOLDINGS LTD` would fail its own pipeline. `replaceAll` is case-insensitive.
- **XML escaping.** `R&D Partners` is stored in OOXML as `R&D Partners`. Asking the verifier about the unescaped form gets the honest answer *the term was never in the original* — true of the raw bytes and useless. `presentRenderings` expands each term to the forms actually present in the original and verifies **all** of them are absent. This is strictly stricter, and a term with no rendering present anywhere still produces the verifier's refusal, which is what stops a caller inflating a clean report with terms the document never contained.

3.6 The status change, and its exact limit

Article 18 moves **INTEGRATION_GAP → ASSURANCE_GAP**.

`Called` is now true and earned: `cmd/veriqo-runtime-evidence` runs the pipeline over a real compressed workbook and appends the disclosure event to the same ledger as the case chain, leaving `AUDIT-014-redaction.derivative_released` behind as committed runtime evidence.

It did not move further, and the reason is recorded in the matrix source. `Qualification` stays `false`:

> "Assessed, not merely run" would require somebody to have concluded the control is complete, and it is not: the workers REFUSE the structures where redacted content most plausibly survives rather than handling them. Refusing is the safe behaviour and it is the right behaviour, but a control that declines a large part of its own problem space has not been assessed as adequate, and marking it so to reach a nicer verdict is exactly the move this matrix exists to prevent.

What remains outstanding, named plainly: no adversarial recovery lab outside VERIQO has attempted reconstruction, and the hard PDF structures are declined rather than solved.

---

4. The authority quintuple

The review's philosophical point was the sharpest thing in the document:

> Kita harus selalu bisa menjawab: **Who is allowed to say this?** Bukan hanya: What does the object contain? Karena source provenance tidak sama dengan decision authority.

`pkg/ontology/authority.go` gives every one of the **40** canonical objects five separable answers:

```
TYPE     what kind of authority this is
SUBJECT  who holds it
BASIS    what confers it
SCOPE    what it reaches
TIME     when it may be exercised
```

Seven authority types, chosen so that the distinction the review cared about is structural rather than editorial:

Type	Meaning
`ACQUISITION`	the right to bring something in from outside — says nothing about what may be concluded
`REGISTRY`	the right to move an object through its lifecycle
`DERIVATION`	the right to compute by a fixed rule — adds no judgement, which is why it can be code
`EPISTEMIC`	the right to say what the evidence supports — the scarce one
`ADJUDICATIVE`	the right to adopt a conclusion and act — held by a person, never the system
`CUSTODIAL`	the right to hold and hand over, with no right to alter or interpret
`DECLARATORY`	the right to state a fact VERIQO records rather than determines

4.1 The tests that make it more than a table

- **`TestProvenanceAndAuthorityAreSeparate`** — `ObjectDocument` carries `ACQUISITION`; `ObjectFinding` carries `EPISTEMIC`. A document from Lloyd's List is impeccably sourced and concludes nothing. The test fails if those two ever collapse into one type.
- **`TestTheImplementationIsNotABasis`** — `Validate` refuses *"the code allows it"*, *"no restriction is enforced"*, *"by convention only"*. A basis like that is how an implementation detail becomes a right, and a declaration missing its basis is the most dangerous shape available: it reads as an authority while recording no reason anyone has one.
- **`TestNoObjectClaimsUnboundedAuthority`** — an authority with no scope bound and no time bound is ownership, not authority.
- **`TestOnlyPeopleHoldAdjudicativeAuthority`** — an adjudicative subject beginning `veriqo/` fails. That would be the system adjudicating, which Article 16 forbids.
- **`TestEveryCanonicalObjectDeclaresItsAuthority`** — all 40, complete. An object whose authority is undeclared is one nobody has had to think about, and it will be the one a dispute turns on.

---

5. Authority uniqueness, proven for code not yet written

The review put the risk exactly:

> Kalau suatu hari developer baru menambahkan `func BuildFinding(...)` ke graph package, kita kembali ke masalah yang sama. Anti-duplication perlu berlaku bukan hanya sekarang, tetapi future-proof.

Every previous anti-duplication test was written about the call graph as it stood. `test/architecture/authority_uniqueness_test.go` parses the module and enforces rules over its *shape*.

5.1 The call rule

No **library** package may call an authoritative constructor — `proof.Seal`, `proof.NewFinding`, `proof.Authorize`, `proof.Decide`. `cmd/` binaries and tests may, because something has to invoke `Seal` for anything to be sealed; a command is a program a human ran, while a library is something other code composes with unknowingly.

5.2 The producer rule, and the mistake worth recording

The first version of these rules keyed on **function names** — anything called `Resolve`, anything called `Append`. It failed immediately, and correctly: the module contains identity resolution, entity resolution and evidence-API resolution, none of which resolves a case, and it contains four separate hash-linked structures.

Tuning that rule until the noise stopped would have produced a rule that caught nothing. So the rules key on the **returned type** instead:

- **ONE CASE RESOLUTION AUTHORITY** — exactly one function in the module produces a `casefabric.Outcome`.
- **ONE LEDGER AUTHORITY** — exactly one produces an `audit.AuditRecord`.

A second discrimination was needed: returning a type is not producing one. Accessors return a stored value, error paths return the zero value, wrappers forward what an authority gave them. `constructsNonEmpty` requires a **composite literal with at least one field set** — because `return Outcome{}, err` is how every fallible function in the module spells "no outcome", and counting those would make every such function look like an authority.

On the ledger, a claim deliberately not made. The module holds several hash chains — the event contract chain, the write-ahead log, the evidence store. Each is real and each is legitimate; they are storage and transport, not the audit ledger. Claiming "one hash chain in VERIQO" would be false, so the rule is stated over the canonical audit record instead.

5.3 The negative proofs

An architecture test that has never failed is a hypothesis. Four tests build a synthetic module containing the violation and check the scanner sees it:

- `TestTheScannerCatchesAGraphPackageThatMintsFindings` — the reviewer's scenario, written out literally as a `caseproofgraph.BuildFinding` calling `proof.NewFinding`.
- `TestTheScannerDoesNotFlagACommandThatDrivesTheChain` — the exemption is real, not an accident of the same scan.
- `TestTheScannerCatchesASecondOutcomeProducer`.
- `TestTheScannerIgnoresAZeroValueReturn`.

---

6. One commercial case across three domains

The review asked for a single real matter rather than thirty-five sources: *"Mulai dengan satu end-to-end commercial case… MARITIME + COMMODITY + INSURANCE… Kalau satu case nyata berhasil, baru scale."*

`test/integration/single_commercial_case_test.go`, **`TestOneCommercialCaseAcrossThreeDomains`**.

The existing `TestSystemIntegrationProofForEveryDomain` runs six domains **separately**, which proves no domain has its own engine. It does not prove a single case can carry evidence from more than one domain at once — which is what a real commercial matter is. A cargo damage claim is simultaneously maritime (where was the vessel, what was the weather), commodity (what was loaded, in what condition) and insurance (what does the policy cover).

One case, `XD-CARGO-2026-001`, carries all three:

Domain	Evidence	Source
maritime	vessel positions and reported weather over the laden voyage	`ais-provider`
commodity	the pre-loading survey recording cargo condition	`independent-surveyor`
insurance	the discharge survey recording the damage	`appointed-adjuster`

Through the full chain: rights before disclosure → provenance and temporal standing → trust → gated absence → reverse proof → qualification → seal → finding → authorized decision → resolution → graph → ledger → redacted disclosure → replay.

Three things it does that the per-domain proof does not:

1. **Rights are evaluated before contact.** The appointed adjuster may view the discharge survey; a broker granted existence-only is refused, and the refusal is asserted.
2. **The weak source is carried, not dropped.** The adjuster is appointed by the insurer, so the discharge survey is party-mediated. That travels into the proof object as a stated limitation.

`TestTheCrossDomainCaseCarriesItsContradictionForward` fails if it is removed — a case that quietly drops its weakest source looks stronger than it is.

3. **The disclosure is real.** The redacted derivative for the broker goes through the Article 18 pipeline, so it is verified rather than asserted.

6.1 What this is, precisely

This is the rig. Every stage is the production code path and the case is genuinely cross-domain. **The evidence is a fixture.**

That distinction is the entire L3/L4 boundary and it is not narrowed here. Running this rig on real AIS positions, a real survey report and a real policy wording is L4, and it requires a data agreement that does not exist. What this establishes is that when such an agreement exists, **there is somewhere for the data to go** — which was not previously true, because nothing composed the three domains onto one case.

`TestTheCrossDomainCaseStatesItsFixtureBoundary` fails if that paragraph is deleted from the file.

---

7. Audit position

	Before this round	After
OPEN	2	2
INTEGRATION_GAP	1	**0**
ASSURANCE_GAP	21	**22**
EXTERNAL_QUALIFICATION	6	6
QUALIFIED	0	**0**

The only movement is Article 18, INTEGRATION_GAP → ASSURANCE_GAP, on the grounds set out in §3.6.

INTEGRATION_GAP reaching zero means **no article now has code that nothing calls**. It does not mean the articles are satisfied — twenty-two of them are ASSURANCE_GAP, which is a statement that nobody outside VERIQO has examined them.

Articles 9 (zero-knowledge proofs) and 15 (operational neutrality) remain OPEN and were not touched.

QUALIFIED remains zero, and no external blocker changed status — not the eight production blockers, not `EXTERNAL_TSA`, not `INDEPENDENT_ASSURANCE`. Nothing in this round could change them, because every one requires somebody who is not VERIQO.

Nothing claims above L3. The maturity model is unchanged: the internal ceiling is `L3_INTEGRATION_VERIFIED`, and L4 upward all require an outside party.

---

8. Verification

`scripts/verify.sh`, full run:

```
== 1/6  go build ./...                                  ==  pass
== 2/6  go vet ./...                                    ==  pass
== 3/6  gofmt (must produce no output)                  ==  pass
== 4/6  zero-external-dependency invariant              ==  pass
== 5/6  go test ./... -race -cover                      ==  pass
== 6/6  race-repeat on consensus-critical packages (5x)  ==  pass
ALL CHECKS PASSED.
```

The redaction workers are written against `archive/zip`, `compress/zlib`, `encoding/xml` and `regexp` — all standard library — so the zero-external-dependency invariant holds with the new format handling in place. No PDF or Office library was added.

New this round:

- `pkg/evidence/redaction/worker` — the three workers, the inspectable view, the fail-closed pipeline
- `pkg/ontology/authority.go` — the quintuple for 40 objects
- `test/architecture/docs_traceability_test.go` — the documentation rule
- `test/architecture/authority_uniqueness_test.go` — the five authorities, AST-proven, with four negative proofs
- `pkg/proof/finding_invariant_test.go` — the four-part Finding invariant
- `test/integration/single_commercial_case_test.go` — the cross-domain case

8.1 Defects found in my own work this round

- **The PDF cross-reference rebuild searched for `"xref"` and found `"startxref"`**, which sits *after* the trailer. An off-by-one-keyword that produced a file no reader could open. Caught because the round's own happy-path test failed, not by inspection.
- **The XML-escaping problem in §3.5** surfaced as a test failure reporting that `R&D Partners` "was never in the original" — which was true of the bytes and revealed that the whole term-matching approach needed the rendering expansion.
- **The first authority rules keyed on names**, flagged four unrelated `Resolve` functions and four unrelated `Append`s, and had to be rebuilt around return types (§5.2).
- **`gofmt` silently reflowed a struct field across lines and my string-replacement no-op'd** — for the third time in this engagement. Caught by grepping for the result rather than trusting the edit.

---

9. What this round does not claim

- The redaction workers handle the formats they accept and **refuse** the structures where remnants most plausibly survive. Refusing is safe; it is not the same as having proven those structures can be redacted.
- No adversarial recovery lab outside VERIQO has attempted reconstruction from a derivative this pipeline produced.
- The cross-domain case is the rig, on fixtures. It is not L4.
- The authority quintuple is a structural argument backed by tests. No independent assessor has read it.
- Nothing in VERIQO is QUALIFIED. Nothing is above L3.

The next boundary is unchanged and is not another architectural round: real rights-aware data under a real agreement, and then an assessor who is not VERIQO.